

Efficient GPU parallelization of adaptive mesh refinement technique for high-order compressible solver with immersed boundary

S. Zaghi^{a,*}, F. Salvatore^{b,2}, A. Di Mascio^{c,3}, G. Rossi^{d,4}

^a CNR-IAC, Istituto per le Applicazioni del Calcolo “Mauro Picone”, Via dei Taurini 19, Rome, Italy

^b CINECA, HPC Department, Via dei Tizii 6/B, Rome, Italy

^c UNIVAQ, Department of Industrial and Information Engineering and Economics, University of L'Aquila, Italy

^d Intel Corporation Italia S.p.A., Via Santa Maria Valle 3, Milan, Italy

ARTICLE INFO

Keywords:

Adaptive mesh refinement
Immersed boundary method
CFD
HPC
GPU
WENO

ABSTRACT

A new, highly parallelized, adaptive mesh refinement (AMR) library, equipped with an accurate immersed boundary (IB) method for solving the compressible Navier–Stokes system is presented. The library, named ADAM, is designed to efficiently exploit modern exascale GPU-accelerated supercomputers and it is implemented with a highly modular structure in order to make easy to leverage it for a wide range of CFD applications. The structured Cartesian grids at the basis of (octree) AMR approach allows to implement very high order accurate models retaining a low computational cost and high level of parallelization. The accurate IB method coupled with efficient AMR technique enables the simulation flows with complex (possibly moving and deforming) geometries. The library is applied to the simulation of a strong shock diffraction over a solid sphere and a detailed discussion concerning the physical results and the parallel performance obtained is presented.

1. Introduction

Nowadays, Computational Fluid Dynamics (CFD) constitutes a fundamental research tool for analyzing very complex phenomena: extremely sophisticated and accurate mathematical and numerical models are able to predict and to discover novel, interesting results in both physical science and engineering applications. In particular, the compressible Navier–Stokes equations (possibly coupled with specific sub-models and modifications) are ubiquitous in all fluid-dynamics phenomena, ranging from low velocity, incompressible hydro-dynamics to hypersonic compressible gas-dynamics. The Navier–Stokes equations are a non-linear, unsteady Partial Differential Equations (PDE) system for which a general, integral solution is not known: the accurate, approximate solution of such systems requires high temporal and spatial resolution, therefore an extraordinary amount of computational resources that only modern supercomputers are able to provide. Consequently, the present CFD research frontier is focused on High Performance Computing (HPC), in order to develop efficient and highly parallel solvers: the most accurate results currently known in literature

have been obtained exploiting exascale (10^{18} floating point operations per second) computing resources, it being its current performance target for top research groups involved in CFD studies. Pointed these out, the challenge in CFD studies has now at least two sides: the continuous improvement of mathematical and numerical models (MNM) and the efficient exploitation of the huge, parallel computing resources provided by modern supercomputers. While MNM development has been always a key feature of CFD, the efficient HPC exploitation is reaching interest especially during the last decades when new heterogeneous computing cluster have been realized. As a matter of facts, many of the most performant supercomputers are heterogeneous systems, based on Graphical Processing Unit (GPU) accelerators coupled with Central Processing Units (CPU): their non-uniform architecture, strongly different from traditional homogeneous CPU clusters, poses new challenges in developing exascale-tailored solvers. The requirement of efficient exploitation of both CPU and GPU based supercomputers is an emerging research field in CFD.

* Corresponding author.

E-mail addresses: stefano.zaghi@cnr.it (S. Zaghi), f.salvadore@cineca.it (F. Salvatore), andrea.dimascio@univaq.it (A. Di Mascio), giacomo.rossi@intel.com (G. Rossi).

¹ Ph.D., Aerospace Engineer, Research Scientist at CNR-IAC.

² Ph.D., Mechanical Engineer, HPC Engineer at CINECA.

³ Ph.D., Mechanical Engineer, Full Professor at UNIVAQ.

⁴ Ph.D., Aerospace Engineer, Application Software Engineer at INTEL.

1.0.1. Towards exascale with adaptive mesh refinement

Historically, Adaptive Mesh Refinement (AMR) techniques have been investigated in order to efficiently use the limited computational resources, see [1–8]. The efficient usage of the computational resources is of particular interest for highly multi-scale phenomena and the modern challenge is to implement it effectively into exascale HPC facilities. AMR approach consists into the dynamical grid adaptation to the flow in order to cluster high resolution where the crucial phenomena happen: the computational resources are thus efficiently adapted to the dynamics of the flows studied. The adaptive nature of AMR approaches is particularly attractive for both complex, moving geometries flows and for unsteady flows characterized by strong multi-scale features. The grid adaptation is obtained by the dynamical refinement or de-refinement of a region of the computational domain: the adaptation criterion is arbitrary and, generally, it has fluid dynamics nature (e.g. the local value of the density gradient is widely used for gas dynamics studies) as well as geometrical one (e.g. the local approximated representation of solid walls). Generally speaking, the regions adapted are cells or blocks of cells and the refinement/derefinement ratio is generally a power of 2 (i.e. 2^1 , 2^2 and 2^3 for one-dimensional, two-dimensional and three-dimensional models, respectively).

The state of the art of AMR solvers counts many block-based implementations. Henry de Frahan et al. [9] develop PeleC, an adaptive mesh refinement solver for compressible reacting flows based on the open source AMReX C++ library, tailored to achieve exascale performance on GPU-bases supercomputers. Hoppe et al. [10] have realized the ALPACA suite, a level-set based sharp interface multiresolution solver for hyperbolic and parabolic conservation laws entailing a multi-phase model and being based on block-adaptation. Freret et al. [11] published a high-order finite-volume based anisotropic-AMR solver developed for magneto-hydrodynamics flows parallelized for CPU-based HPC facilities (see also [12]). Similarly, Owen et al. [13] presented a finite-volume AMR solver for multi-species flows characterized by high multi-scale features.

1.0.2. Immersed boundary methods

Another attractive feature of AMR technique is to strongly improve the accuracy of Immersed Boundary methods (IBM) that are themselves extremely convenient to simulate complex geometries with low computational costs. It is well known that structured Cartesian grids are extremely easy to be parallelized, enabling cutting-edge computing performances as well as allowing the implementation of very high order accuracy numerical methods. On the contrary, body-fitted, general curvilinear grids are more computational expensive and high quality grids are time-consuming and difficult to realize. However, structured Cartesian grids can represent complex (possibly moving and deforming) geometries with only very low details, strongly degrading the whole simulation results accuracy. To overcome such limits, IBM approaches (among others) have been developed and widely investigated in the last decades, see [14–17]: the key idea is to “immerse” complex geometries into the Cartesian grids and to modify the fluxes computation of the cells boundaries where the geometries “fall”, either by a diffuse-interface forcing term or by a sharp-interface cell-cutting tracking (see [18]). The complex geometries are not directly meshed, they are rather immersed into the Cartesian grids, retaining all their attractive features. One of the main limitations of Cartesian grids equipped with IBM is the inability to selectively cluster high resolution in the proximity of geometrical boundaries: the AMR technique is able to overcome this limitation, enabling the high resolution refinement with dynamical adaptation to the flow conditions and to the geometrical shapes (possibly moving and deforming).

1.0.3. Manuscript outline

In the present manuscript we present an efficient implementation for highly parallel GPU-based supercomputers of an AMR library entailing accurate IBM solver (see [19]), that is designed with a high level of modularity for a wide range of applications. A detailed application to a compressible gas dynamic phenomenon is also provided, in order to assess the accuracy and the performance of the library. The manuscript is organized as following: in Section 2 the library software architecture and its efficient parallelization are described; in Section 3 a brief physical description of the gas dynamic test case studied is presented, within the mathematical and numerical models used to simulate it; in the same section a detailed assessment of the performance obtained is provided as well as the discussion concerning the physical results; lastly, in Section 4 some concluding remarks are summarized.

2. ADAM framework: Application code description

2.1. Software architecture

The software architecture of modern Computational Fluid Dynamics codes aims to combine the requirements of code generality and maintainability with those of achieving maximum performance. From the point of view of generality, the fundamental goals of a CFD software include the possibility of adding new numerical methods, or even new equations, without compromising the general structure of the code, and expanding the choices the user can make by configuring the simulation input files. From the perspective of performance maximization, the requirements take the form of a software’s ability to adequately exploit modern High Performance Computing (HPC) hardware architectures. Prominent among modern HPC architectures today is the general purpose GPU trend initiated by NVIDIA [20,21], and more recently followed by AMD [22], and soon by Intel GPUs [23]. Programming paradigms associated with the use of such resources are unfortunately still varied, evolving and only partially interoperable: CUDA (C and Fortran) paradigm for NVIDIA boards, the HIP (C language only) paradigm for AMD boards allow for optimal code performance but require major code rewriting starting from a CPU code, while the standardized OpenMP [24,25] and OpenACC [26] paradigms are relatively simpler to implement but the performance achieved may be more limited. Other paradigms are less widespread but may potentially gain popularity in the future. Nowadays, some CFD applications that are excellent with respect to algorithm generality [27] are lacking with respect to the ability to exploit modern GPU-based technologies. On the other hand, higher-performance applications even in the HPC domain [28–30] have much more limited algorithm maneuverability margins.

The approach presented in this paper is based on the following cornerstones: multi-block grid with identical Cartesian rectilinear blocks composed by equispaced cells, with the possibility to locally increase the resolution by means of block-based Adaptive Mesh Refinement (AMR) algorithms; multi-node parallelization based on Message Passing Interface (MPI).

The software architecture, on the other hand, allows generality, through the modular design, both with respect to the equation to be solved and with respect to the computational back-end.

Generality should be understood as minimization of the replicated code that would be needed to write entirely different codes for each equation and for each computational back-end, i.e., $N_{\text{equations}} \times N_{\text{backends}}$. In this framework, we distinguish 4 types of code section:

- code sections independent of both the equation and the computational back-end, such as handling the AMR grid and Input/Output management;
- code sections dependent on the equation but not on the computational back-end, such as field or numerical method initializations;
- code sections dependent on the computational back-end but not on the equation, such as updating ghost regions between blocks;

Equation \ Backend	Independent	Dependent
Independent	Parameters Grid Tree Field Adam	<code><equation>/equation_base</code>
Dependent	<code>Base_<backend></code>	<code><equation>/equation_<backend></code> <code><equation>/main_<backend></code>

Fig. 1. Main code components grouped according to the equation or back-end dependency.

- code sections dependent on both the equation and the computational back-end such as calculating a convective or viscous flux.

Modular design thus allows code of each type to be combined so as to avoid duplication for different equations or computational back-ends. The conceptual diagram of the code design is summarized in Fig. 1. A concise description of the role of each of the components follows:

- *parameters*: contains some constants used by different procedures in the code
- *grid*: contains the basic elements of the grid of the individual block
- *tree*: contains information about the AMR hierarchy of blocks, indexing of blocks for load balancing, and procedures for preparing the refine-derefine in the AMR hierarchy
- *field*: contains the CPU version of the evolved field of the simulation with a generic number of variables and blocks, maps for MPI communication, and procedures for refine/derefine fields according to AMR requirements
- *adam*: coordinates *field* and *tree* for AMR management, handles input/output of fields and snapshots for restart
- *base_<beta>*: given a *<beta>* backend, contains the evolved field in the back-end oriented version (e.g. field on GPU memory), MPI buffers for the back-end, performs copies between the field contained in field and the back-end oriented field, ghost cell exchange routines (which include any MPI communication)
- *<alpha>/equation_base*: given an equation *<alpha>*, contains the auxiliary variables for the equation solved in the CPU version, the initialization routines for fields and numeric methods, the variables and routines for statistics
- *<alpha>/equation_<beta>*: given an equation *<alpha>* and a back-end *<beta>*, contains the routine that is called directly from the main program, and all implementations of the numerical solution algorithms that work on the variables allocated on the back-end
- *<alpha>/main_<beta>*: given an equation *<alpha>* and a backend *<beta>*, is the main program.

2.2. AMR implementation and parallelization

The code is implemented in Fortran, following the Fortran 2003 object style. Each component described in the previous section corresponds to a derived data type that includes variables and type bound procedures.

A crucial point for the efficient implementation of the described structure is to properly devise the memory structure for grid management and in particular the AMR. Fig. 2 represents a summary diagram of motivations and implementation of objects in this regard, aspects that will be briefly discussed in this section. Basically, the grid is a

three-dimensional multi-block with rectilinear Cartesian blocks. Each block consists of the same number of equispaced cells, and the physical variables are meant to be defined at the cell center. In addition to its own cells, each block includes some external layers (ghost layers) used to compute terms that require values on cells contiguous to the considered cell. The ghost layers extend into the 3 dimensions in a full way, i.e., including planes, edges and corners for a total of 26 ghost regions for each block (6 planes, 12 edges, 8 corners): this allows supporting numerical schemes in which contiguous regions are needed in the 3-dimensional sense. Multi-process parallelization is handled by MPI, with each process capable of processing a dynamically varying number of blocks, as will be described shortly.

Initially, there is a single block that corresponds to the whole numerical domain of the simulation, which is thus a parallelepiped. AMR is based on the fact that each block can split into 8 identical *sibling* blocks that replace the original block, thus realizing a local refinement of the overall grid. Similarly, eight *sibling* blocks can rejoin and regenerate the *parent* block from which they originated. Each block is then characterized by a level that corresponds to the number of splits of the initial block required for its creation, and a triplet of coordinates (bx,by,bz) with respect to the (potential) grid of blocks at that level (the range of these coordinates clearly depends on the level itself). The resulting structure is the classical octree.

The AMR hierarchy is managed by the derived type *tree* and must have a high degree of flexibility to allow rapid insertion and destruction of octree components: therefore, a hash-map (dictionary) was used, allowing elements – corresponding to blocks – to be added and deleted from the dictionary in O(1) times. The keys of the dictionary are integers (64-bit) that linearize the 4 coordinates (level-bx-by-bz) of the block. The extraction of the dictionary value given a certain key is handled by an auxiliary derived type *tree_bucket*, while each dictionary value contains all the features of the block of interest is handled in another auxiliary derived type called *tree_node*.

To achieve adequate levels of mesh refinement with a reasonable total amount of grid points, the single block is expected to have a low number of cells and is therefore computationally inefficient to process individually during the actual computation (typically a system of differential equations), especially with respect to modern heterogeneous parallel architectures characterized by a very high number of computing cores (of the order of 10 K). For this reason, in the computation phase, the loop on the hash-map must be replaced by a more efficient construct and memory structure. Therefore, a derived type *field* was introduced to manage the physical field *q* evolved by the solver; *q* is organized as an array with 5 indices: 3 spatial indices corresponding to x/y/z, an index related to the physical quantity considered (e.g., density, velocity along x), and a fifth index representing the block index. For the structure to be efficient, the block index must represent existing blocks with contiguous values and thus cannot be the index used for the hash-map. A mapping system makes it possible to switch from the hash-map index to the field block index and vice versa.

For the most part, the computation phase proceeds without having to know the details of the AMR hierarchy. However, when working on a certain block, it is necessary to know the positioning of the block as the physical coordinates of its points, as well as the indices of the contiguous blocks for ghost exchange (or any boundary condition to be imposed). For this reason, some additional arrays are prepared during the AMR grid update phase, to provide this information efficiently while avoiding full hash-map access.

Overall, as mentioned above, such a field structure is extremely simple and suitable for massive parallelization featured by modern GPU architectures. However, GPU computations normally require operating on memory allocated specifically to GPUs. The most efficient and popular paradigms (CUDA for NVIDIA, HIP for AMD) for computation require that the management of this memory is explicitly in the charge of the programmer. For this reason, the code architecture ensures that

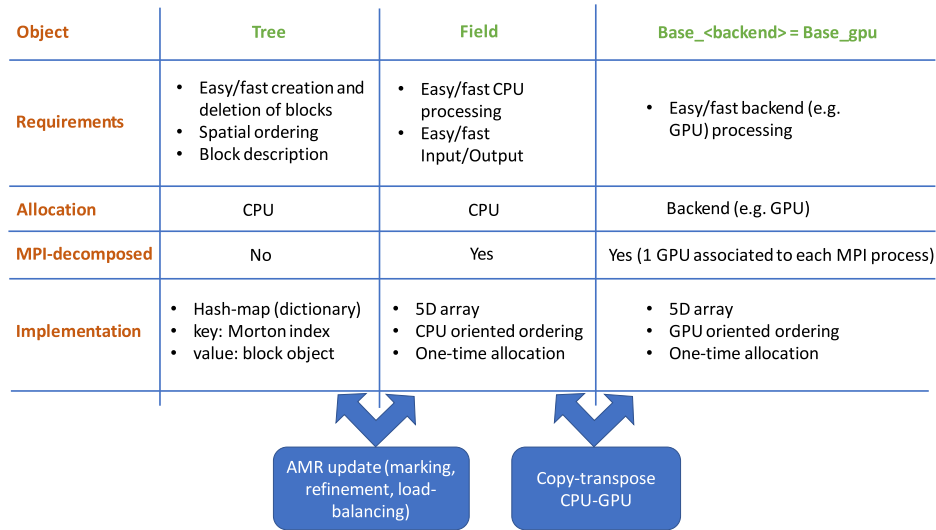


Fig. 2. Diagram of main features of AMR grid management in the implemented object-based code architecture. The two boxes in the bottom represent how the content of connected objects is synchronized.

the computations are not performed directly on the array q contained in $field$, but on a specialized array for the considered computation backend – which we call $q_backend$ – that resides in backend memory, for example on the GPU or even on the CPU if the backend is CPU-based. Generally $q_backend$ is allocated once at the beginning of code execution, so that it occupies the entire GPU memory, and q is allocated accordingly of the same overall size, but this behavior can be altered in the implementation part of the equation. The currently implemented backend is based on CUDA Fortran and is therefore suitable for using NVIDIA GPUs. In this case, $q_backend$, which we will henceforth call q_gpu features transposed index ordering with respect to q : specifically in q_gpu the block index – normally the largest index – is the contiguous index in memory (the first index in Fortran) in order to maximize access data coalescing for contiguous CUDA threads. The q_gpu array is part of $base_gpu$ derived type since it is independent of the equation being considered except for the number of variables being passed to initialization. The ghost MPI communication procedures are also contained in $base_gpu$ since they are independent of the solved equation. Overall, the design described allows the entire computation phase to be performed on the GPU without the need for CPU–GPU copies that would dramatically slow performance. The routines that deal with CPU–GPU copies – which also include data transposition to synchronize q and q_gpu – are called only during input/output operations and during AMR structure updates. In the implementation of the vast majority of the compute core there is in fact no evidence that this is AMR code, because all AMR management is delegated to external objects implemented independently of the equation. For the implementation of CUDA kernels, automatic “cuf” kernels, that allow high code readability, are mostly used but some explicit kernels were also needed where more advanced CUDA features are required. Implementation of other back-ends is under development, but is not expected to pose structural significant challenges in the conceived architecture.

From the parallelization point of view, it is worth noting that the structure of q_gpu in $base_gpu$ is effective with respect to a data-parallel approach by assigning each MPI process a different subset of the blocks to be allocated and processed. However, it is necessary to balance the load per process efficiently, i.e., by imposing spatial contiguity between the blocks allocated on the same process, so as to minimize ghost exchanges between blocks on different processes. In this regard, the load-balancing mechanism now implemented is based on Morton’s space-filling curves. As far as $tree$ is concerned, on the

other hand, the content of the octree is shared among all processes and at the moment the implementation of related operations is essentially serial: this is certainly a limitation of the current implementation but a possible parallelization of tree operations as well will be the subject of future developments.

AMR update is one of the most complex steps in a code of this type and occurs according to 3 main steps:

Block requirement marking :

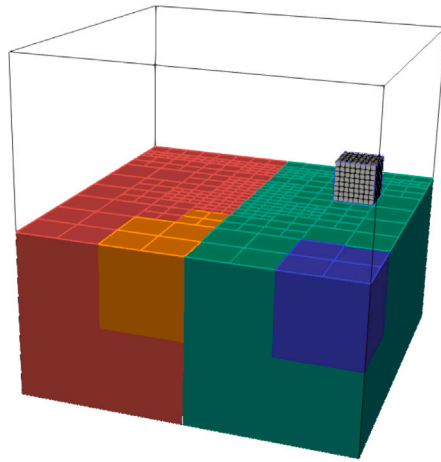
- (a) During the computation phase, a routine (executed by the computation back-end) evaluates and marks each block, according to some criteria, such as the gradient level of a certain variable, with respect to its need to be split, be recomposed, or remain as is.
- (b) A sanitizing procedure (performed on CPU) corrects the markers assigned to each block according to consistency criteria, i.e., (a) for re-grouping, the eight blocks sharing the same parent must all exist (not split) and be marked as “to be re-grouped”; (b) the AMR level between two contiguous blocks must differ by at most one; (c) the maximum AMR level must not exceed the maximum allowed level imposed by input.

Refinement/coarsening :

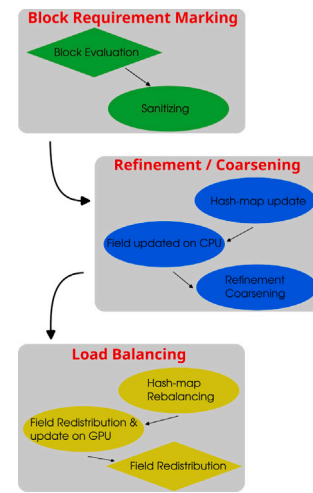
- (a) The hash-map in $tree$ containing the AMR structure is updated according to the expected refinement/derefinement requirements. Maps are generated that will be used by the $field$ to perform refinement and coarsening of the marked blocks.
- (b) The $field$ array is copied to its CPU version (q_gpu is copied to q). The $field$ performs refinement and coarsening, by interpolation, of the marked blocks. At the end of this phase, the load balancing of the various processes is no longer optimal.

Load balancing :

- (a) The hash-map rebalancing is performed at the level of the octree structure redistributing blocks with respect to MPI parallelization. Each block is assigned to a new process according to a contiguity criterion, which, as mentioned, in



(A) Sketch of simulation domain: AMR structure of blocks is shown in the horizontal plane whereas different colors (red, orange, green, blue) represent the MPI process owning the block according to Morton load-balancing. An example block (colored in gray) with its cells is also shown.



(B) AMR update algorithm flow chart.

Fig. 3. AMR update and load balancing algorithms.

the current implementation is based on Morton curves [31, 32].

- The field is redistributed both locally and among different MPI processes to arrive at the final balanced and efficient configuration as contiguity. The field array is copied to its back-end version (q is copied to q_gpu).
- The maps that allow the exchange of ghosts and other auxiliary arrays for each block are regenerated.

In Fig. 3(a) an example of AMR structure and parallel decomposition is shown.

The refinement and coarsening criteria depend on the field values in the blocks and thus on the equation being solved. It is recommended that the criterion chosen avoid flip-flop outcomes so criteria should be provided that encourage persistence of a fixed grid resolution. For example, if the criterion for splitting a block depends on exceeding an S_{split} threshold value, the re-grouping criterion should be based on a $S_{recompose} < S_{split}$ threshold value so as to create a grid persistence zone between $S_{recompose}$ and S_{split} .

Updating of ghost nodes includes three steps: using values from blocks belonging to the same MPI process, using values belonging to different MPI processes, using values derived from boundary conditions. For the first two cases, interpolations may be required if the communicating blocks are not at the same AMR level. Second-order interpolations are currently implemented. For the currently implemented CUDA Fortran back-end, MPI communications for block updates exploit CUDA-Aware MPI to avoid explicit copies between GPU and CPU variables to be exchanged.

Finally, the Input/Output is realized by exploiting HDF5 (each process writes its blocks to a different file) coordinated by an XDMF file that easily allows the use of visualization software on the produced HDF5 files.

3. Test case: Strong shock wave diffraction by a sphere

To demonstrate the capabilities of ADAM framework, we develop a compressible Navier–Stokes solver application (named NASTO hereafter) exploiting the GPU back-end of ADAM library. A shock wave

propagation phenomenon is chosen as the main benchmark,⁵ it being an interesting phenomenon for both computational and physical reasons:

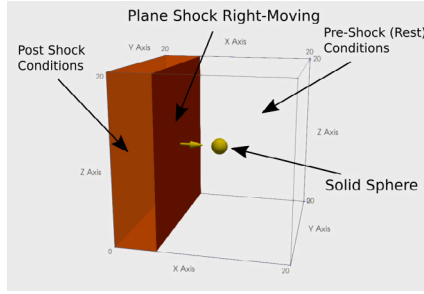
- the unsteady, non linear interaction/propagation of shock waves (and discontinuities, in general) generates extremely complex flow fields patterns, where smooth regions are interspersed by discontinuous ones; this kind of complex, irregular, unsteady flow fields constitutes a challenging benchmark for AMR methods where grids adaptivity is exploited at its maximum level; as a matter of facts, it constitutes a highly multi-scale, unsteady phenomenon;
- shock waves interaction is a crucial phenomenon in applied physics: ranging in environmental, industrial and military applications, the study of shocks propagation is ubiquitous, see [33–39].

Mathematical and numerical studies are often focused on 2D cases [40–42], however to stress grids adaptation to its limits, a pure 3D case has been chosen, namely the strong shock wave diffraction by a solid sphere as the one studied experimentally by Bryson and Gross [43]. Fig. 4 reports a schematic representation of the numerical setup of the diffraction simulation: the computational domain is made by a cube of 20 non-dimensional units per axis and a solid sphere is placed in [10, 10, 10]. A plane shock (the incident shock) at Mach 2.89 is moving from left to right, the post/pre shock conditions are reported in Fig. 4(b). At the left side of the domain a supersonic inflow boundary conditions (equal to post conditions) is imposed while an extrapolation outflow is imposed at other domain sides. The fluid (air) is assumed to be a perfect, Newtonian fluid which physical properties are also reported in Fig. 4(b).

3.1. Mathematical and numerical methods

The fluid dynamics of the shock diffraction phenomenon considered is accurately described by the Navier–Stokes equations. Based on the

⁵ A simplified benchmark is reported into [Appendix](#).



(A) Computational domain setup.

	Post shock conditions	pre shock conditions
density	5.1432014	1.4
pressure	9.0454500	1.0
velocity [x,y,z]	[2.0451168, 0, 0]	[0, 0, 0]
fluid properties		
specific heat at constant pressure $c_p [\frac{J}{kg \cdot K}]$	1005.2	
specific heat at constant volume $c_v [\frac{J}{kg \cdot K}]$	718.0	
dynamic viscosity $\mu [\frac{kg}{m \cdot s}]$	0.0013485123	
thermal diffusivity $k [\frac{W}{m \cdot K}]$	1.83178995	

(B) Physical fluid properties and post/pre shock initial conditions.

Fig. 4. Shock wave diffraction simulation setup.

original theory of Navier [44] and Stokes [45], see [46,47], the conservation laws implemented into the NASTO application read, in integral, non dimensional form:

$$\frac{\partial}{\partial t} \int_V \bar{U} dV + \oint_{S(V)} (\bar{F}_c + \bar{F}_d) \cdot \bar{n} dS = \int_V \bar{Q} dV \quad (1)$$

V being the control volume, $S(V)$ its boundary, $\bar{n}$ the outward unit normal of $S(V)$ and where $\bar{U}$ is the vector of conservative variables, $\bar{F}_c$ is the convective part of the fluxes tensor, $\bar{F}_d$ is the diffusive part of the fluxes tensor and $\bar{Q}$ is the source terms vector. For fluid not subjected to external forces and heat sources, the above vectors and tensors could be written as:

$$\bar{U} = \begin{bmatrix} \rho \\ \rho \vec{v} \\ \rho E \end{bmatrix} \quad \bar{F}_c + \bar{F}_d = \begin{bmatrix} \rho \vec{v} \\ \rho \vec{v} \vec{v} + p \bar{I} - \frac{\vec{\tau}}{Re} \\ (\rho E + p) \vec{v} - \frac{\vec{\tau} \cdot \vec{v}}{Re} - \frac{k \nabla T}{Pr Re} \end{bmatrix} \quad \bar{Q} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad (2)$$

The conservative vector $\bar{U}$ and the fluxes tensor $\bar{F} = \bar{F}_c + \bar{F}_d$ are non dimensional quantities where ρ is the density, $\vec{v}$ is the velocity vector, E is the specific, total energy, T is the absolute temperature, p is the static, isotropic pressure ($\bar{I}$ being the identity tensor), $\vec{\tau}$ is the viscous shear stress tensor, k is the thermal conductivity coefficient ($-k \nabla T$ being the Fourier's law of heat conduction). The non dimensional form is obtained through the non dimensional numbers of Reynolds and Prandtl, namely $Re = \frac{\rho_0 U_0 L_0}{\mu_0}$, $Pr = \frac{\mu_0 c_p}{k_0}$, where the quantities noted with the subscript "0" are the dimensional reference values. The viscous shear stress tensor, in general case, is defined as:

$$\tau_{ij} = \mu (\partial_i v_j + \partial_j v_i) + \lambda (\nabla \cdot \vec{v}) \delta_{ij} \quad (3)$$

where μ is the dynamic viscosity of the fluid, λ is the second viscosity and δ_{ij} is the Kronecker delta $\delta_{ij} = \begin{cases} 0, & \text{if } i \neq j \\ 1, & \text{if } i = j \end{cases}$. For a Newtonian fluid the dynamic viscosity and the second viscosity are related $\lambda = -\frac{2}{3}\mu$. The system (2) is completed by the constitutive equation of state that for a perfect gas is $p = \rho R T$ where R is the fluid constant: $\gamma = \frac{c_p}{c_v}$, $R = c_p - c_v$, $c_p = \frac{\gamma R}{\gamma - 1}$, $c_v = \frac{R}{\gamma - 1}$. The other equations of state are: $E = c_v T + \frac{|\vec{v}|^2}{2} = \frac{p}{\rho(\gamma - 1)} + \frac{|\vec{v}|^2}{2}$ is the total specific energy definition, $a = \sqrt{\frac{\gamma p}{\rho}}$ is the speed of sound (acoustic velocity).

To solve system (2) NASTO exploits a finite difference scheme in which the integral form is rewritten into the following differential one:

$$\frac{\partial \bar{U}}{\partial t} + \frac{\partial (\bar{F}_c + \bar{F}_d)}{\partial x_i} = 0 \quad i = 1, 2, 3 \quad (4)$$

where the repeated index syntax is used. The system (4) constitutes a Partial Differential Equations (PDE) system where both hyperbolic (convective) and parabolic (diffusive) terms are present. Exploiting the method of lines [48] the time and space operators are solved separately. Firstly, an approximation of the fluxes divergence is found at each cells center and then the solution is evolved in time with the appropriate Ordinary Differential Equations (ODE) solver.

The building block of the spatial operator is the approximation of cell fluxes. Without loss of generality, let us consider a 1D case. Considering a generic flux $f(x)$, the space operator is approximated by

$$\frac{\partial f(x)}{\partial x_i} = \frac{h_{x+h/2} - h_{x-h/2}}{h} \quad (5)$$

where h is the grid spacing along x , that is constant in the uniform Cartesian AMR blocks of ADAM framework, and $h(x)$ is a high order approximation of fluxes, see [49] and the reference there cited for proof and details. The NASTO application uses a WENO reconstruction method for obtaining high order approximations of cells fluxes, see [49–52]. The proper upwind decomposition necessary for the hyperbolic terms is obtained by the classical flux vector splitting attributed to Lax and Friedrichs (also known as Rusanov method), see [53–55].

After the WENO reconstruction and the fluxes computation, the system of equations (4) constitutes a system of ODE that can be solved by means of a multi-stages Runge–Kutta method, originally developed by Runge [56] and Kutta [57]. An explicit scheme belonging to the Strong-Stability-Preserving family, SSP, see [58], is implemented into NASTO application. In the Butcher's form, see [59], the scheme reads:

$$U_i^{n+1} = U_i^n + \sum_{k=1}^{N_s} b_k K_{i,k} \quad (6)$$

where N_s is the number of Runge–Kutta stages used and the K_k is the k th stage defined as:

$$K_k = Dt \cdot R \left(U_i^n + \sum_{l=1}^{N_s-1} a_l K_{i,l} \right) \quad (7)$$

The NASTO application implements a 3 stages, 3rd order SSP Runge–Kutta scheme, see [60–67] for details.

The system of PDEs solved is strongly non-linear, thus there is no general stability criterion ensuring the convergence: it is usual to adopt a heuristic approach in which the results of linear stability analysis are applied to non-linear case. A CFL-like condition, see [68], is used in order to provide a domain-of-dependence criterion used as an upper bound for a stability coefficient. In particular, in a three dimensional frame of reference the following sufficient, local (cell), condition is adopted:

$$CFL_i = \frac{\max(|u_i| + a_i, |v_i| + a_i, |w_i| + a_i)}{\min(\Delta x_i, \Delta y_i, \Delta z_i)} \Delta t_i \quad (8)$$

where u_i, v_i, w_i , are the velocity vector components of the i th cell and where a_i , Δt_i and $\Delta x_i, \Delta y_i, \Delta z_i$ are the local speed of sound, the local time step and the cell dimensions, respectively. In an unsteady simulation all the transitory phenomena must be accurately captured, therefore a global limit is computed as a minimum of the above local conditions:

$$CFL = \min_i (CFL_i) \quad (9)$$

Table 1

Summary table of runs considered for performance evaluation. Inputs: number of cells in each direction for the single block N_{xyz} , required size of cells for coarsest blocks δ_C , required size of cells for finest blocks δ_F , ratio between δ_C and δ_F , number of GPUs used N_{GPUs} . Outputs: maximum number of blocks obtained N_{blocks} , maximum number of cells (millions) N_{cells} , elapsed time for one iteration (without AMR) T , elapsed grind time for one iteration $T_g = T/N_{cells} * N_{GPUs}$, elapsed time for one AMR update T_{AMR} , total elapsed time T_{TET} . Runs 3 and 10 actually correspond to the same simulation but are repeated for clarity purposes. The performance are measured at the physical time $t=5$, when the maximum number of blocks/cells are generated, i.e. corresponding to the most complex and computational demanding phase of the simulation.

#	N_{xyz}	δ_C	δ_F	R_δ	N_{GPUs}	N_{blocks}	N_{cells}	$T[s]$	$T_g[s]$	$T_{AMR}[s]$	$T_{TET}[h]$
Scaling vs. N_{GPUs}											
1	8	0.5	0.125	4	1	7008	3.6	1.019	0.28	4.25	0.61
2	8	0.5	0.125	4	2	7008	3.6	0.477	0.27	3.11	0.33
3	8	0.5	0.125	4	4	7008	3.6	0.221	0.25	2.36	0.18
4	8	0.5	0.125	4	8	7008	3.6	0.114	0.25	2.1	0.15
5	8	0.5	0.125	4	16	7008	3.6	0.0716	0.32	1.91	0.20
6	8	0.5	0.125	4	32	7008	3.6	0.0541	0.48	1.84	0.25
Scaling vs. N_{xyz}											
7	16	0.25	0.0625	4	8	7729	31.7	0.629	0.16	2.96	0.65
8	32	0.125	0.03125	4	64	8513	279.0	0.925	0.21	4.3	2.66
Scaling vs. R_δ											
9	8	0.5	0.25	2	1	1562	0.8	0.163	0.20	0.74	0.06
10	8	0.5	0.125	4	4	7008	3.6	0.221	0.25	2.36	0.18
11	8	0.5	0.0625	8	16	32446	16.6	0.275	0.26	6.2	1.64
12	8	0.5	0.03125	16	64	163745	83.8	0.396	0.30	36.47	9.63
13	8	0.5	0.015625	32	256	684258	350.3	0.67	0.49	527.97	25.21

3.1.1. Immersed boundary method coupled with AMR grids

In order to simulate complex (moving) bodies, the ADAM framework is equipped with an Immersed Boundary method (IBM). The IBM implemented is a recent modification of classical IBM approaches and is described in details in [19].

The method is based on the definition of a signed distance function, whose zero level identifies the body boundary, like the level-set approach of Sethian and Smereka [69]. The signed distance function is used to extrapolate the solution outside the flux domain. The solution in the external points is then modified in order to obtain the correct flux on the bodies boundaries. This approach benefits from local adaptive mesh refinement, but special attention must be done to the algorithm coupling IBM with AMR technique.

The first step of the algorithm is to obtain an extrapolation of the solution inside the body. Let us introduce a signed distance function $\phi(x, y, z)$: the modulus of this function is equal to the distance from the body boundary, whereas its sign is chosen to be positive inside the body and negative in the flow domain. The body surface is identified as the contour $\phi = 0$ of the function; moreover, the unit vector to each contour $\phi = \text{constant}$ can be computed as $\mathbf{n} = \nabla\phi/|\nabla\phi|$; in particular, $\mathbf{n}$ coincide with the inside unit vector to the body on the contour $\phi = 0$. The function $\phi(x, y, z)$ easily identifies the points inside the flow domain and those inside the body, where a special treatment is needed to enforced the boundary conditions. The implemented IBM starts from the solution extrapolation inside the body; this extrapolation is performed by searching the steady state solution with respect to a pseudo time τ of the equation:

$$\frac{\partial \mathbf{q}}{\partial \tau} + \mathbf{n} \cdot \nabla \mathbf{q} = 0 \quad (10)$$

here $\mathbf{q}$ is a generic set of solution variables, e.g. primitives or conservative variables, and $\mathbf{n}$ is the above introduced distance function normal versor. Solving the steady state of Eq. (10) (namely solving the eikonal equation associated) in the flow field points adjacent to the body boundary $\phi = 0$ means to propagate the solution along the normal direction of ϕ to the points inside the body, i.e. to extrapolate the solution inside the body. As reported in [19], a single propagation step with CFL close to 1 per Runge–Kutta sub–stage is sufficient for a proper extrapolation.

Once obtained the (eikonal) extrapolation of the solution inside the body, the fluxes of cells where the body boundary $\phi = 0$ falls can be properly computed accordingly to the physical boundary conditions of the body, e.g. viscous wall, inviscid wall, etc. However, when AMR technique is coupled with this IBM and with *ghost cells* technique

for boundaries exchange between blocks, special attention must be placed in the order of operations. As a matter of facts, in NASTO application the coupled AMR-IBM integration algorithm consists of, for each integration time step:

- update AMR grids *if necessary* (case-specific requirement) using the current time solution:
 - adapt grids accordingly to refinement/de-refinement criteria;
 - recompute signed distance ϕ ;
- for each Runge–Kutta stage computation step:
 - update ghost cells of solution $\mathbf{q}$ for non immersed boundaries;
 - for each eikonal extrapolation step:
 - * extrapolate solution (evolve eikonal) inside body (where ϕ changes sign);
 - * update ghost cells of solution $\mathbf{q}$ for non immersed boundaries;
 - use extrapolation to impose immersed boundary conditions (in cells where $\phi = 0$ falls);
 - compute corrected residuals (space operator) using non IBM and IBM fluxes;
 - compute Runge–Kutta stage using corrected residuals.
- integrate solution on one time step;

Using the above described mathematical and numerical methods, the shock diffraction by sphere phenomenon has been simulated with a range of different grids, comparing different resolutions and different AMR strategies. In the next Sections 3.2 and 3.3, the measured computational performance and physical results are presented with detailed comments.

3.2. Performance results

The evaluation of performance, and particularly parallel performance, in an AMR code requires considerable attention: different measures are possible with respect to both the type of scaling considered and the different quantities that can be measured; the measures are also clearly related to the considered test case. From this point of view, the test case of Bryson's shock diffraction on the sphere presents significant

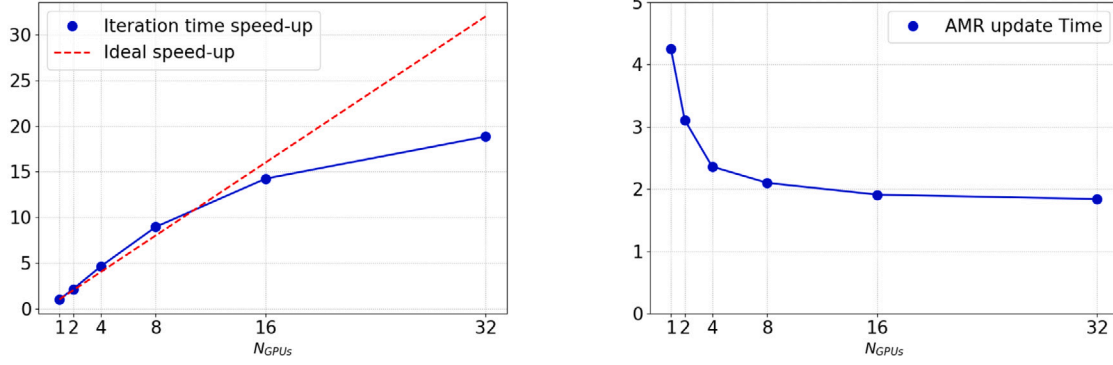


Fig. 5. (Left) Strong scaling speed-up of iteration elapsed time against number of GPUs. The measured speed-up is compared to the ideal speed-up. Points correspond to runs 1, 2, 3, 4, 5, 6 in Table 1. (Right) AMR update time for the same simulations.

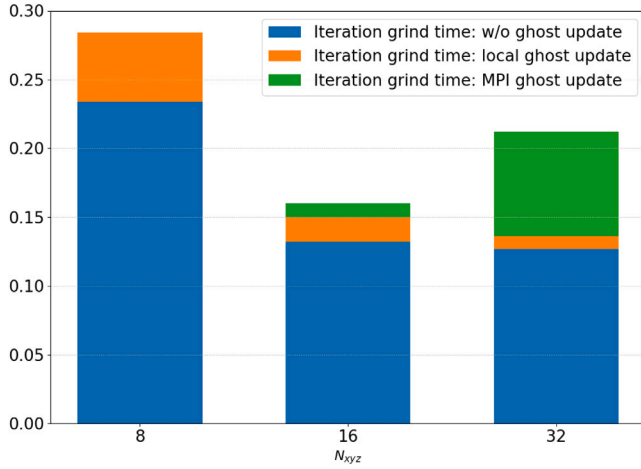


Fig. 6. Scaling of iteration grind time against the number of points of each block composing the AMR grid structure. As the number of points of each block is doubled, the imposed coarse and fine resolutions δ_C and δ_F are halved so that the resulting AMR structure is similar. Number of GPUs is multiplied by eight so that the load of each GPU is nearly constant. However, since the AMR structure is not identical, grind time are reported instead of simple iteration times. In each bar, the iteration grind time is decomposed in three contributions: (1) the iteration grind time of local ghost update (orange) corresponds to ghost update involving blocks owned by the same MPI process, (2) the iteration grind time of MPI ghost update (green) corresponds to ghost update involving blocks owned by different MPI processes (MPI communications are performed), (3) the iteration grind without ghost update (blue) is the remaining part of elapsed iteration time.

computational challenges, in particular with respect to grid resolution requirements. First, the complex shock structures require, in addition to the use of the numerical shock capturing methods previously recalled, fine grid resolution in different regions of the field. Second, the highly transient nature of the test case, requires refinement in zones that vary widely as the simulation evolves: from the beginning to the end of the simulation, most of the zones of the field must be refined at least in a certain time interval, but the location of the zones to be refined while time is advanced is not known accurately a priori. In short, the physical case represents a considerable stress-test for a compressible CFD AMR code. We report in this section the performance results measured using the CUDA Fortran back-end capable of using NVIDIA GPUs. Specifically, the GPUs used are the V100 in the Marconi100 cluster at Cineca. The AMR thickening criterion is based on the value of the density gradient modulus and two required grid resolutions δ_C (minimum) and δ_F (maximum). We will consider three types of analysis:

1. strong scaling: same setup except the number of computational units (GPUs).

2. (weak) scaling against the number of cells per block: as the number of cells is doubled, both δ_C and δ_F are halved in order to yield an (approximately) constant AMR structure. At the same time the number of GPUs is increased by a factor of 8 to keep the computational load per GPU roughly unchanged.
3. (weak) scaling against the ratio of δ_C to δ_F : this is the most AMR-oriented analysis where a very large number of blocks may be generated. The number of GPUs used is increased so that the computational load per GPU is kept approximately constant.

All results discussed in this section refer to simulations described in Table 1. The results reported and discussed are extracted under the conditions of maximum number of blocks in a given simulation.

The first performance measure we report concerns as mentioned strong scaling, i.e., performance analysis for a fixed numerical case against the computational units, i.e., the number of GPUs. The reference case is represented by a grid with $N_x = N_y = N_z = N_{xyz} = 8$ blocks, AMR configured with $\delta_C = 0.5$ and $\delta_F = 0.125$. A maximum of 7008 blocks with a total of about 3.6 million cells are produced during the simulation. The results are shown in Fig. 5 considering a range between 1 and 32 GPUs. On the left figure, the speed-up of elapsed iteration time is shown considering iterations that do not include AMR grid updates. The speed-up is close to ideal up to 8 GPUs – even superlinear – while at 16 GPUs (case where each GPU evolves about 225 K cells) there a slight degradation is visible but still a good efficiency (speed-up divided by ideal speed-up) close to 90% is achieved. At 32 GPUs, on the other hand, the speed-up degrades significantly with an efficiency below 60%. The rough conclusion from shown data is that the code is efficient if each GPU has a mesh portion greater than 250 K cells, a reasonable result considering the parallelization – due the number of CUDA threads – requirements of a V100 GPU in order to express its computational power.

The Figure on the right shows the AMR update times as the number of GPUs varies. As expected, the scalability of this type of computation is much more limited, given the type of involved operations (e.g., serial octree updates and redistribution of load per process to optimize load balancing). The impact of AMR update times on overall simulation times is a complex issue concerning the type of test case considered. As mentioned above, this test case requires relatively frequent AMR update so AMR update times plays an important role, but, in cases where the grid resolution requirements are more stable over time, AMR update may be performed rarely or even only in the early stages of the simulation.

The second performance measure involves comparing similar grids from the AMR point of view, but composed of blocks of different sizes. Starting from the strong scaling reference case with $N_{xyz} = 8$, we specifically consider grids with blocks of $N_{xyz} = 16$ and $N_{xyz} = 32$, respectively. To produce the same AMR structure, the δ_C and δ_F levels are halved by moving from one grid to the next one. Correspondingly,

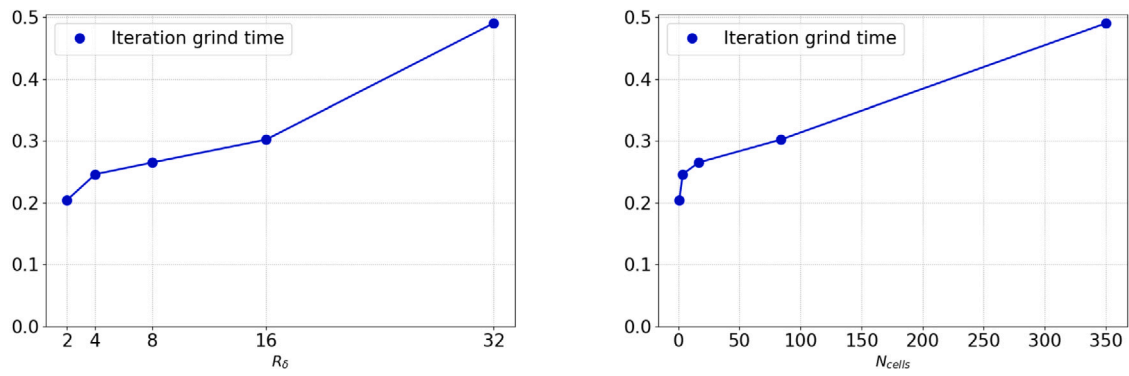


Fig. 7. Scaling of iteration grind time considering fixed δ_c and doubling the ratio R_δ . Grind time is plotted against R_δ (left) and against the total number of cells (right).

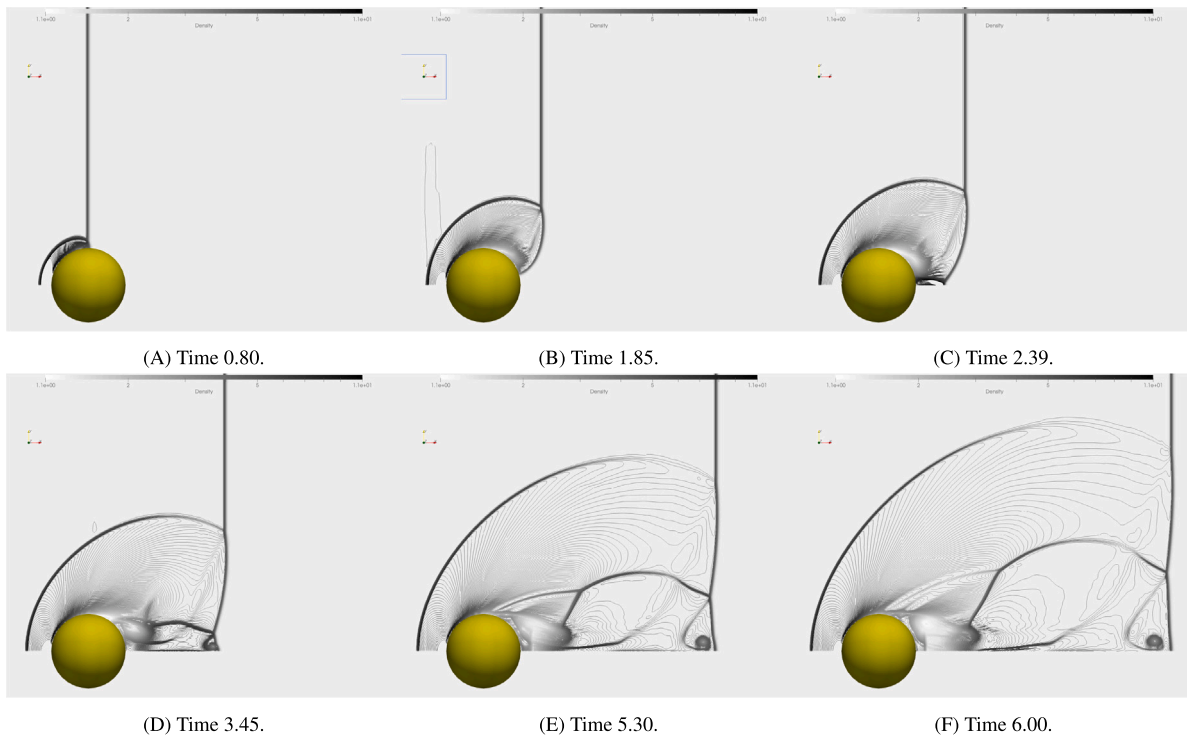


Fig. 8. Shock wave diffraction: time (non dimensional) evolution of density flow field; run #12, $\delta_F = 0.03125$.

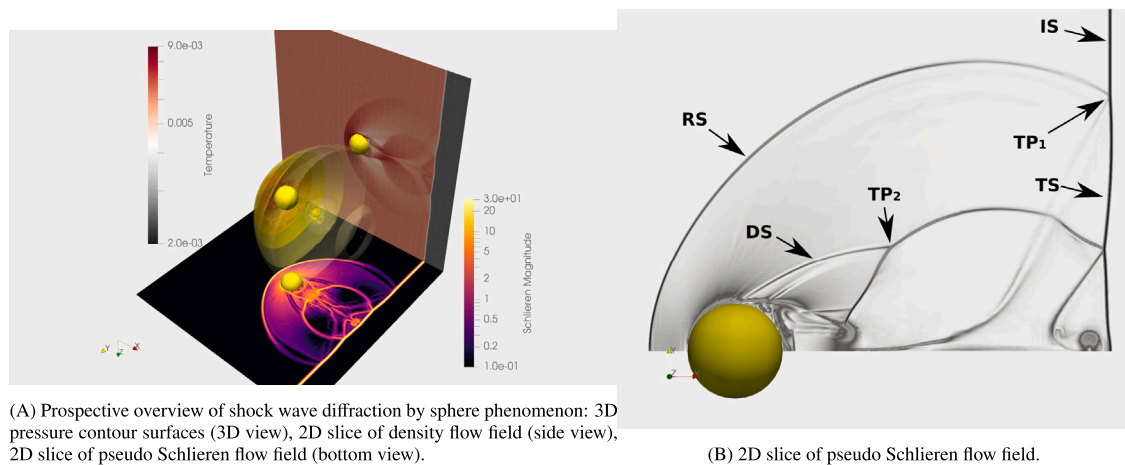


Fig. 9. Main flow field structures formed after shock wave diffraction by sphere: ; acronyms meaning: IS Incident Shock, RS Reflected Shock, DS Diffracted Shock, TS Transmitted (Mach) Shock, TP Triple Point.

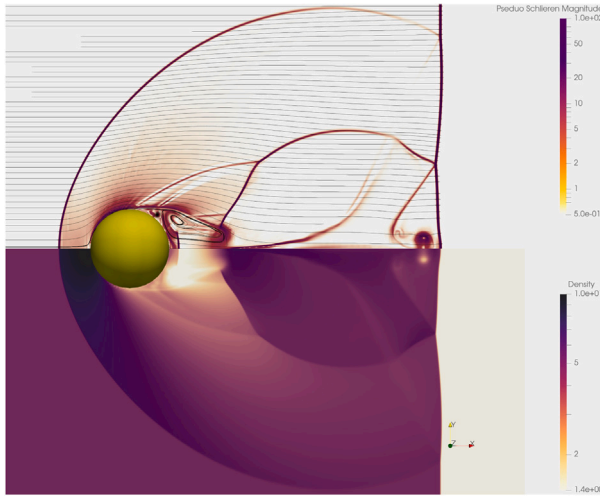


Fig. 10. Shock Wave Diffraction by a sphere: pseudo Schlieren (top) with plane stream traces and density (bottom) flow fields.

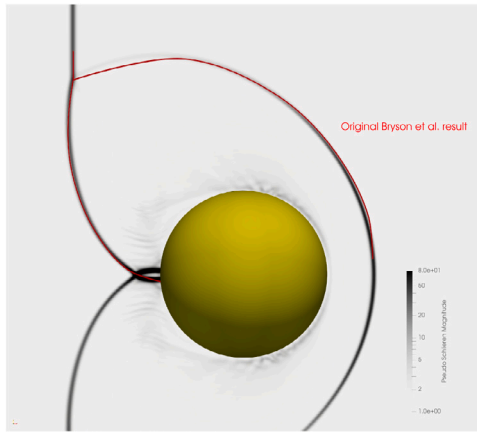


Fig. 11. Qualitative comparison between solution of grid #4 ($\Delta f = 0.03125$) and the experiments of Bryson et al.

the GPUs used for computation are increased to 8 and 64 respectively to keep the GPU load roughly constant. The results show that, as expected, the obtained grids are very similar from the AMR point of view to the reference grid but are not identical. Therefore, for comparison, it seems more correct to use grind time, i.e., time per time-step normalized with the number of cells per GPU: $T_g = T / (N_{cells} / N_{GPUs})$. The values of T_g for the 3 cases are shown in Fig. 6. To better understand these values, the times were decomposed into the component due to ghost updates between blocks of the same process, the component due to ghost updates between blocks of different processes, and the remaining portion of time, mainly related to the actual computation. Comparing the case of the reference grid with the second grid, it is observed that the grind time is significantly reduced. In fact, the emergence of MPI communication times is modest and more or less compensates for the intra-process ghost exchange, which is reduced instead. The most important change is related to the performance improvement of the computation part due presumably to the fact that each GPU is working with a few larger blocks instead of many smaller blocks. Possible low-level insights via profiler could help to understand more deeply the reasons for this change. Concerning the third grid, the computational part remains unchanged, while the increase in MPI communication due to the increase in the number of processes and the number of communications between processes not on the same cluster node becomes significant. However, the grind time is still lower than

the corresponding value of the reference grid. Regarding AMR update times, the values reported in Table 1 grids show modest variations around 3.5 s. This behavior is related to the fact that the AMR structure of the grid is basically unchanged in the 3 cases.

The third performance measure we report is an analysis related to the most advanced use of AMR, where the magnitude δ_C is held constant, while δ_F is progressively reduced to reproduce gradually smaller-scale dynamics. The reference case was now selected as the strong scaling case using 4 GPUs. Under such conditions we have indeed proved that the efficiency of the solver is still optimal. We then consider four more grids, one obtained by doubling δ_F , and three grids obtained by progressively halving δ_F . The first interesting result reported in Table 1 is the number of produced blocks/cells: as Δf is halved, the number of cells increases by a factor of between 4 and 5. This factor should be compared to the factor of 8 that would occur if grid halving were uniform, not adaptive: in summary, the finest grid has 350M cells instead of the 8 billions it would have with uniform refinement. As for the grind time, it is represented in Fig. 7 as the R_δ ratio varies (left) and as the total number of cells varies (right) for the 5 simulations. As expected, T_g worsens as R_δ increases however the increase is less than a factor of 3 while R_δ increases by a factor of 32. This means that the advantage of using this approach to refined the grid is still cost-effective. In contrast, the plot against the number of cells is interesting because, excluding the first point, it shows a fairly accurate linear trend. Since the 5 simulations are performed by increasing the number of GPUs by a factor of 4, while, as mentioned, the grid size increases by a factor between 4 and 5, the loading conditions are not too dissimilar. Thus, the degrading of T_g is presumably related to the increase in the number of blocks and processes, and thus to the intra and extra-process communications of ghost nodes, as shown in the previous analysis.

Another important aspect concerns AMR update times, which unfortunately become dramatically high when the number of blocks increases (T_{AMR} exceeds 500 s for the finest grid). The impact of T_{AMR} in the total simulation time depends as mentioned on the frequency required for remeshing. Such high times are not a problem for statistically stationary simulations in which remeshing in the early stages of the computation is sufficient, but they become a problem for simulations of transient phenomena in which, for example, we want to follow the propagation of a spatial disturbance in the field. For such cases, single block refinement and consequent reduction of the total number of blocks (as shown in the previous analysis) may be an acceptable compromise choice. However, to achieve very fine resolutions the number of total cells easily becomes dramatic with dramatic consequences on memory occupation thus putting at risk the actual ability to execute a certain run. Detailed performance studies of AMR remeshing will be objects of future development to understand the extent to which it is possible to decrease T_{AMR} and thus allows even very high block-count computation to be handled.

3.3. Physical results

The strong shock wave diffraction by a sphere constitutes a complex, unsteady, gas dynamics phenomenon. The analytical, closed solution is unknown and it must be investigated by experimental and numerical studies. As stated in Section 3, the simulation setup is designed to simulate one of the experiments developed by Bryson and Gross [43].

Fig. 8 reports the time evolution of the density flow field: reading the sequence from left to right and from top to bottom, the incident shock (IS) impinges on the solid sphere surface generating a reflected shock (RS, also indicated as bow shock), a diffracted shock (DS) and triple point (TP). While the time advances, this structures evolves and shocklets, material discontinuities and vortical regions emerge in the sphere wake.

Fig. 9 provides an overview of the results obtained from the run 12 in Table 1 ($\delta_F = 0.03125$). In particular, Fig. 9A reports a 3D overview

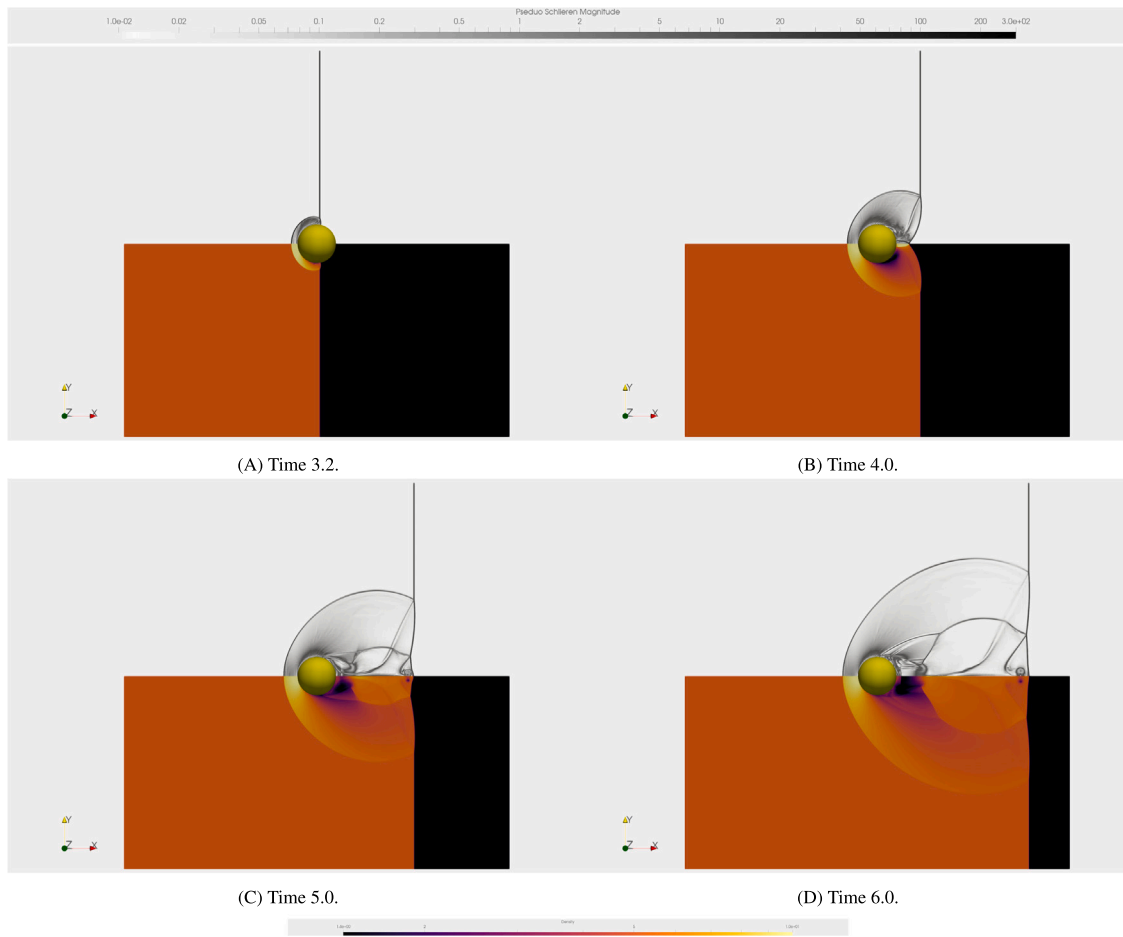


Fig. 12. Shock wave diffraction by sphere: density (bottom) and pseudo Schlieren (top) flow fields; run #13, $\delta_F = 0.015625$.

of the pressure iso-surfaces within the plane projections of density and pseudo Schlieren flow fields in the side and bottom planes, respectively. The pseudo Schlieren flow field is computed as the density gradient and it points out the loci of discontinuities (shocks and material discontinuities). Fig. 9B (results obtained from run 13 in Table 1, $\delta_F = 0.015625$) reports a detailed zoom of 2D slice of the pseudo Schlieren flow field at the time $t = 6$ (in non dimensional time units) with the principal discontinuities locations emphasized by labels. To the authors' knowledge, these results have the finest resolution for the fully three dimensional simulation of strong shock wave diffraction over a sphere. Analyzing Figs. 8 and 9 the complex, unsteady nature of the diffraction clearly emerges. Shock waves are clearly discontinuities of very small scale, whereas the continuous regions of expansion originated during the diffraction evolution have much larger scale.

Fig. 12 reports an analogous time serie for the results of run 13 of Table 1: the pseudo Schlieren field is compared with density flow field in order to clearly point out the small scale of discontinuities with respect the smooth varying regions. The multi-scale nature of the problem is even more evident in Fig. 10 that reports a comparison of the slices of pseudo Schlieren (top slice) over which the stream-traces are reported and density (bottom slice) flow fields. The results are obtained from run 13 of Table 1, $\delta_F = 0.015625$.

Pointed out the high multi-scale and unsteady nature of the flow studied, it is interesting to analyze the grids adaptation results.

Fig. 13 compares the grid adaptation obtained with different Δ_{fine} requirements, having fixed the Δ_{coarse} one. The finer is the resolution target, the higher are the refinement levels exploited and the range of fluid scales efficiently resolved. Conversely, the comparison reported in Fig. 14 is obtained varying also the Δ_{coarse} requirement: comparing

the results *per column* it evident that for a target Δ_{fine} the higher is the Δ_{coarse} , the higher is the levels number exploited with a more *adapted* grid, whereas the lower is the Δ_{coarse} , the more uniform is the grid. This is also emphasized in Fig. 15 where a detailed zoom comparison is reported. Fixing the Δ_{fine} requirement, it is not obvious or generally valid which is the best compromise between the more adapted grid and the more uniform case because other considerations can play an important role. For example, if the computational time, for a certain computational resources, is crucial it seems to be reasonable to use a more uniform grid, limiting the computational time used for grid adaptation. On the contrary, if the computational time is not limit, rather the aim is to use the available computational resources to obtained the finest resolution possible, a more adapted grid seems to be the best strategy.

To the authors knowledge, experimental results of the simulated shock diffraction is not available in literature. As a consequence, the original work of Bryson and Gross [43] has been used to obtain at least a qualitative comparison with experimental results. Fig. 11 presents the pseudo Schlieren flow field (only the upper half of the domain slice) computed by means of the grid #4 ($\Delta_{fine} = 0.03125$) at the time $t = 4$: the diffraction of the incident shock is started, the reflected curved shock is fully developed and the transmitted shock interact with the reflected and incident shocks in the first triple point while diffracted shock and second triple point are not yet formed. This configuration is the same reported in Fig. 7 (plate 5) at page 21 of the original Bryson and Gross [43]. In Fig. 11 the shocks structure photographed by Bryson and Gross has been digitalized (red line) and overlapped on the numerical flow field here presented. The comparison shows a very good agreement: the main discrepancy is constituted by the form and position of the curved

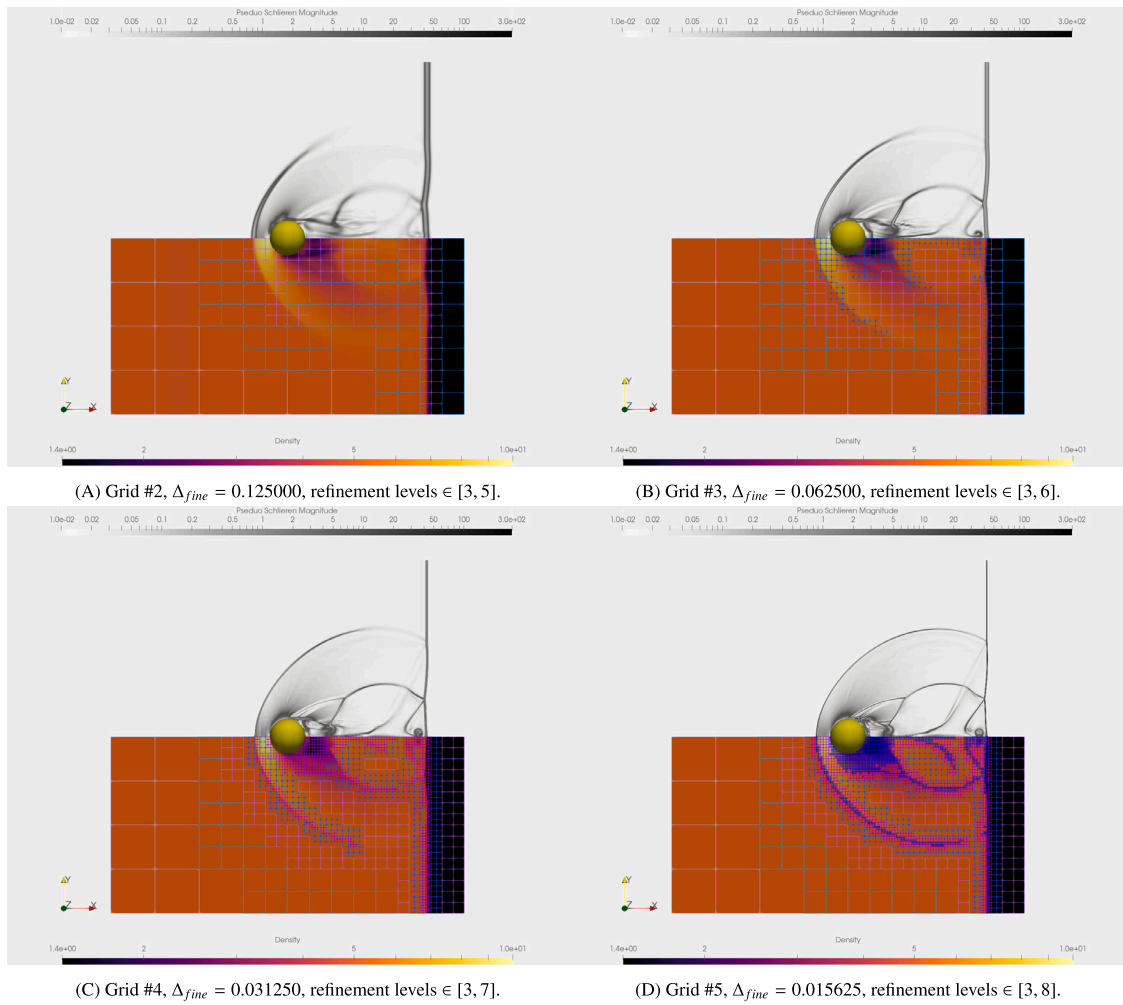


Fig. 13. Shock wave diffraction by sphere: density (bottom, with grid outlines) and pseudo Schlieren (top) flow fields; AMR grids comparison; grids refinement levels $\in [3, 8]$.

reflected shock that in the digitalized experiment seems to be slightly anticipated with respect the numerical results. However, the digitalized curve has large error and only qualitative conclusions can be taken.

4. Concluding remarks

In this manuscript, a new AMR CFD library and application are presented, namely the ADAM software suite. ADAM is built with a multi-block Cartesian grids with the ability to locally and dynamically refine and de-refine by means of an efficient hierarchical data structure, realizing an octree AMR mesh. Within a state of the art AMR capability, ADAM is equipped with novel Immersed Boundary method exploiting the signed distance function to track immersed geometries, see [19], demonstrating the IBM capabilities when coupled with an efficient AMR technique. WENO high order reconstruction methods have been also implemented into the software suite.

ADAM is designed to exploit modern GPU/CPU-based supercomputers by means of Fortran CUDA and MPI. Detailed description of the implementation and parallelization of ADAM have been presented in the manuscript: the library's high modularity shown allows to easy develop new applications for different fluid dynamics phenomena leveraging the library's AMR capability. Moreover, the modular generic programming paradigm used allows to the easy implementation of other back-ends exploiting different types of computing accelerators.

To measure ADAM's parallel performance the strong shock wave diffraction over a solid sphere has been simulated and the results obtained discussed in details. This gas dynamic phenomenon constitutes

a challenging benchmark for AMR solvers, it being characterized by strongly unsteady and extremely multi-scale fluid structures the tracking of which requires huge computational resources. To the authors' knowledge, the results presented have the finest resolution shown in literature for the fully three dimensional diffraction of a strong shock wave over a sphere.

The Fortran CUDA parallelized application test demonstrates a good speed-up on the CINECA's Marconi100 supercomputer: although being a benchmark tailored to Nvidia GPU supercomputer, ADAM is designed to support different *back end* modules to exploit different supercomputers accelerators and different parallelization paradigms, e.g. modules programmed with OpenMP and OpenACC paradigms. Even though the performance measured are generally good, it has been also pointed some area where improvements are necessary, i.e. the serial octree status update after AMR re-meshing.

Following ADAM research activities will concern both new library applications as well as library improvements: the AMR-IBM capabilities will be exploited to simulate aerodynamics phenomena, e.g. to analyze flow fields around airplanes and to predict aerodynamics performances, while the main developing efforts will be focused on introducing new *back end* modules exploiting different parallel programming paradigms and on developing new ADAM's applications (equations). Moreover, the library development will be focused also on the enhancement of the parallel speed-up, in particular on the AMR re-meshing efficiency.

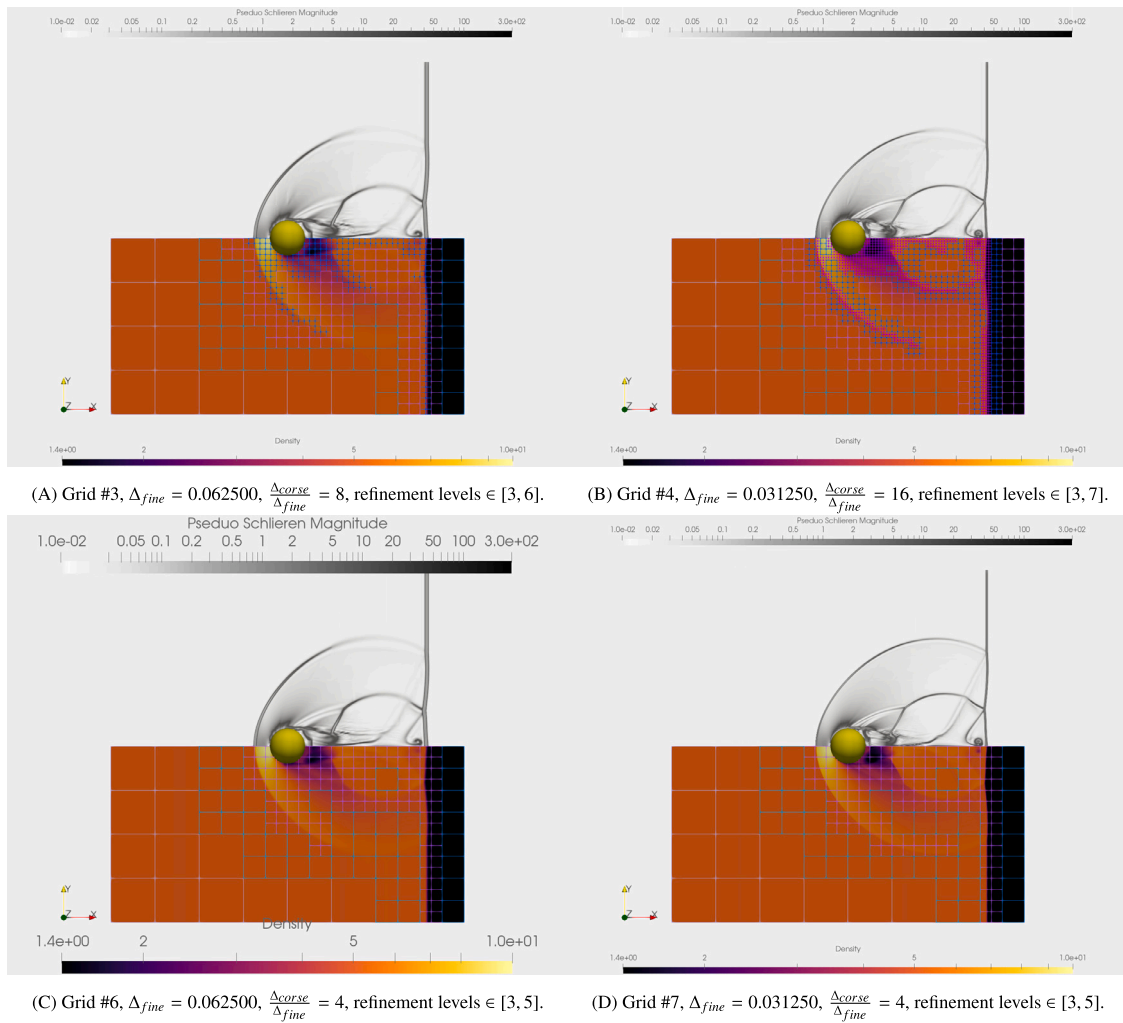


Fig. 14. Shock wave diffraction by sphere: pseudo Schlieren flow field with grids outline; AMR grids comparison; varying $\frac{\Delta_{coarse}}{\Delta_{fine}}$ ratio (grids #3,#4) vs fixed ones (grids #6,#7).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

We acknowledge the CINECA award under the ISCRA initiative, for the availability of high performance computing resources and support.

Appendix. Simplified benchmark

Let us consider a simple test case (see Poinso and Lele [70]) to assess ADAM performance with a less physically complex phenomenon: an isentropic, subsonic moving vortex is simulated into a periodic domain and its motion is tracked for a complete period, i.e. until the vortex reaches the starting position flowing through the periodic boundary. Fig. A.16 reports the numerical setup of this simple test case: the periodic domain is a cube of side $L = 10$ m, and the vortex is initially placed at coordinates $[2, 5, 5]$ m and it is moving towards right at $Mach = 0.1$. The figure reports the stream-traces of the initial velocity

vector over the counter lines of x component in order to emphasize the initial position of the isentropic vortex. The x -boundaries are periodic thus the moving vortex returns to the initial position after a period of $T = 0.29$.

Considering the physical fluid properties reported in Fig. A.16(b), the initial conditions into the computational domain is computed using the following equations:

$$r^2 = \frac{(X - X_v)^2 + (Y - Y_v)^2}{R_v^2} \quad (A.1)$$

$$\rho = \rho_0 \cdot \left(1 - \frac{\gamma - 1}{2} M_v^2 e^{\frac{1-r^2}{2}} \right)^{\frac{1}{\gamma-1}} \quad (A.2)$$

$$p = p_0 \cdot \left(\frac{\rho}{\rho_0} \right)^\gamma \quad (A.3)$$

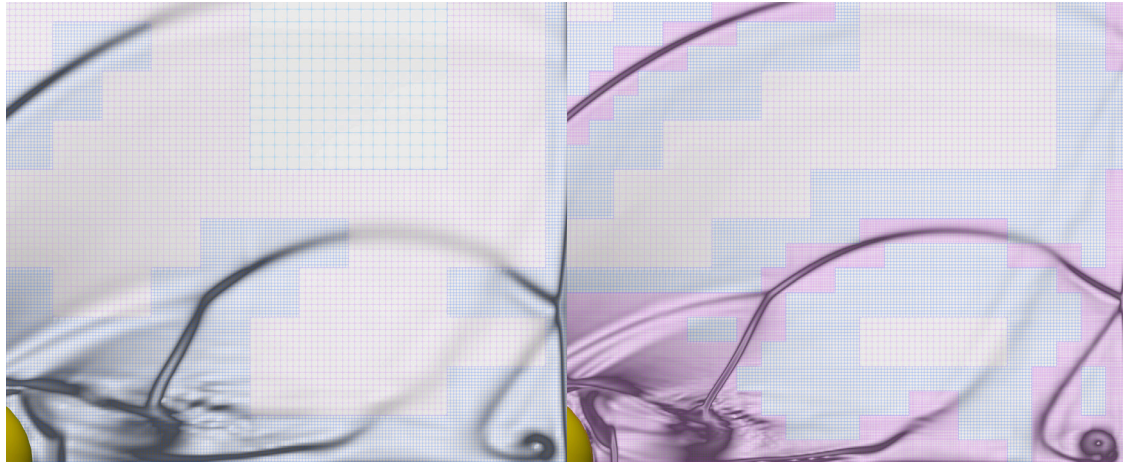
$$u = \left[M_0 - M_v \frac{Y - Y_v}{R_v} e^{\frac{1-r^2}{2}} \right] a_0 \quad (A.4)$$

$$v = \left[-M_v \frac{Y - Y_v}{R_v} e^{\frac{1-r^2}{2}} \right] a_0 \quad (A.5)$$

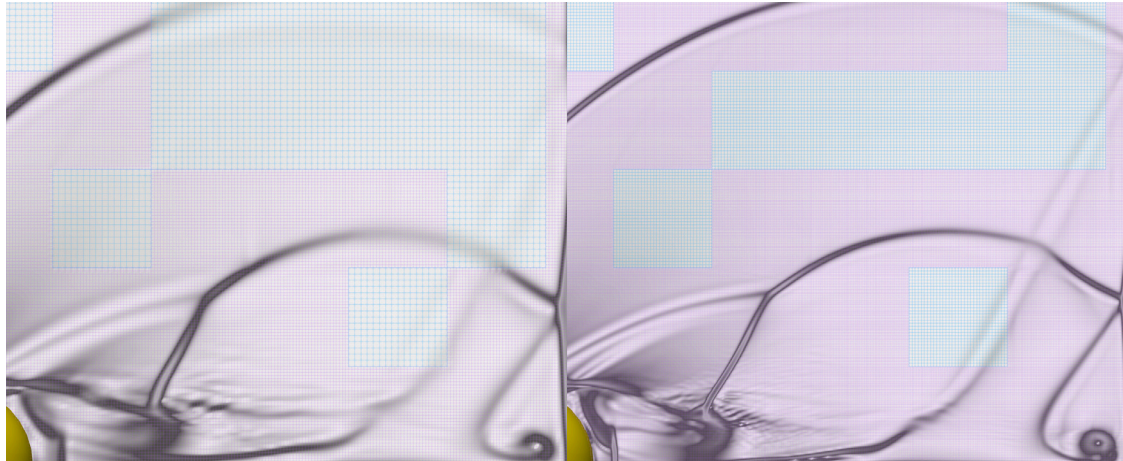
$$w = 0 \quad (A.6)$$

where p, ρ, u, v, w are the density, pressure and xyz-velocity components, respectively.

The AMR grid is adapted to the vortex tracking the gradient of the enthalpy and the maximum refinement level reached 5 corresponding

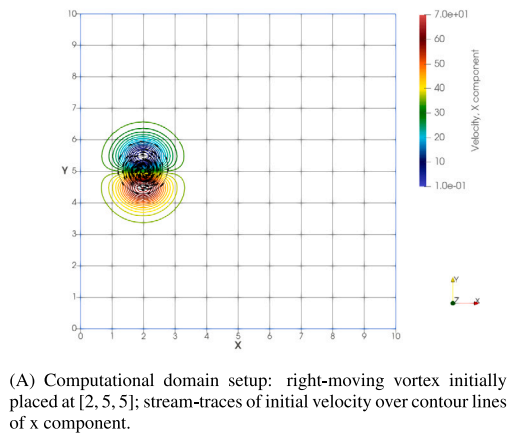


(A) Grid #3, $\Delta_{fine} = 0.062500$, $\frac{\Delta_{corse}}{\Delta_{fine}} = 8$, 16.6M cells, 16#GPUs, refine- (B) Grid #4, $\Delta_{fine} = 0.031250$, $\frac{\Delta_{corse}}{\Delta_{fine}} = 16$, 83.8M cells, 64#GPUs, refinement levels $\in [4, 6]$. refinement levels $\in [5, 7]$.



(C) Grid #6, $\Delta_{fine} = 0.062500$, $\frac{\Delta_{corse}}{\Delta_{fine}} = 4$, 31.6M cells, 8#GPUs, refine- (D) Grid #7, $\Delta_{fine} = 0.031250$, $\frac{\Delta_{corse}}{\Delta_{fine}} = 4$, 278.9M cells, 64#GPUs, refinement levels $\in [4, 5]$. refinement levels $\in [4, 5]$.

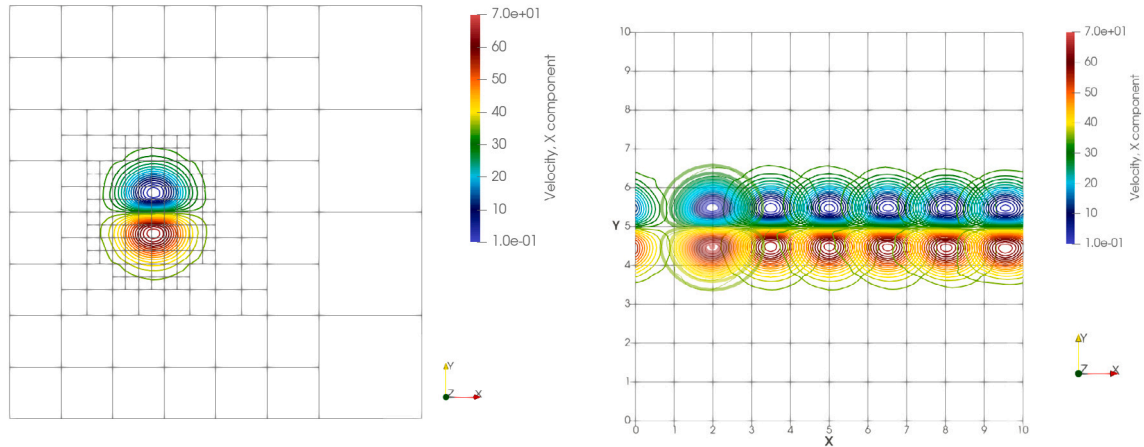
Fig. 15. Shock wave diffraction by sphere: pseudo Schlieren flow field with grids mesh details; AMR grids comparison; varying $\frac{\Delta_{corse}}{\Delta_{fine}}$ ratio (grids #3,#4) vs fixed ones (grids #6,#7).



specific heat at constant pressure $c_p[\frac{J}{kg \cdot K}]$	1005.2
specific heat at constant volume $c_v[\frac{J}{kg \cdot K}]$	718.0
undisturbed density $\rho_0[\frac{kg}{m^3}]$	1.226
undisturbed pressure $p_0[Pa]$	101325
undisturbed speed of sound $a_0[\frac{m}{s}]$	340.15
undisturbed velocity $V_0[\frac{m}{s}]$	[34.015, 0, 0]
undisturbed Mach number M_0	0.1
vortex initial position coordinates $X_v, Y_v, Z_v[m]$	[2, 5, 5]
vortex radius $R_v[m]$	0.5
vortex Mach number M_v	0.1

(B) Physical fluid properties of vortex flow.

Fig. A.16. Shock wave diffraction simulation setup.



(A) AMR grid structure: $\Delta_{fine} = 0.0625$, refinement levels $\in [2, 5]$; contour lines of (B) Vortex flow time sequence over a period: contour lines of velocity x-component, initial position in [2, 5] pointed out in bold.

Fig. A.17. Vortex flow simulation: AMR grid structure and flow time sequence.

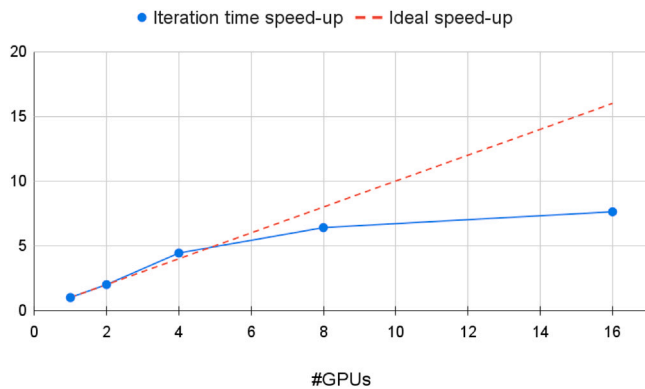


Fig. A.18. Strong scaling speed-up of averaged iteration elapsed time against number of GPUs. The measured speed-up is compared to the ideal speed-up.

to $\Delta_{fine} = 0.0625$. As shown in Fig. A.17(a), the AMR grid has a quite simple structure: because of the simple subsonic convection, the vortex shape remains undisturbed (except for the numerical dissipation due to the WENO3 upwind scheme used), thus also the AMR structure is essentially undisturbed, it being convected within the vortex. As a matter of facts, the AMR grid has essentially a constant number of blocks during the simulation, thus minimizing the AMR adaptation computational overhead and allowing for a clear assessment of the computational performance. In particular the ARM grid reaches a maximum of 4768 blocks (almost constant during the simulation) for a total of about 2.4M of cells.

The Fig. A.17(b) reports a time sequence of the vortex flow for a period, contour lines of x-component of velocity being plotted. The initial position at [2, 5, 5] is emphasized in bold lines: the vortex flow is well tracked, although some smearing is visible. The smearing is essentially due to the numerical diffusion of the WENO3 upwind scheme used. The AMR grid refinements jumps seem to not pollute the solution.

Using the above described test case, a strong scaling analysis has been performed. Due to the limited dimensions of the grid, the strong scaling analysis has been limited up to 16 GPUs on the same MARCONI100 superpc at CINECA facility used for the shock diffraction test case. Fig. A.18 reports the speed-up measured. The measured speed-up has been computed using the *averaged step timing*, i.e. the step timing averaged over all the simulation, thus averaging also the AMR re-meshing time. The measured speed-up is comparable with the strong

scaling analysis reported in Section 3.2: in this simplified test case the AMR re-meshing does not involve a great variation of the number and position of blocks, the AMR structure being a sort of *self similar* moving shape, thus the measured performance are essentially related to only the CFD solver. This benchmarks also points out that the flow smearing across the AMR grids jumps is limited and controlled.

References

- [1] Berger MJ, Olinger J. Adaptive mesh refinement for hyperbolic partial differential equations. *J Comput Phys* 1984;53(3):484–512. [http://dx.doi.org/10.1016/0021-9991\(84\)90073-1](http://dx.doi.org/10.1016/0021-9991(84)90073-1), URL <http://www.sciencedirect.com/science/article/pii/0021999184900731>.
- [2] Berger M, Leveque R. An adaptive cartesian mesh algorithm for the Euler equations in arbitrary geometries. In: 9th AIAA computational fluid dynamics conference, Buffalo, New York, 1989.
- [3] Berger M, Colella P. Local adaptive mesh refinement for shock hydrodynamics. *J Comput Phys* 1989;82(1):64–84. [http://dx.doi.org/10.1016/0021-9991\(89\)90035-1](http://dx.doi.org/10.1016/0021-9991(89)90035-1), URL <http://www.sciencedirect.com/science/article/pii/0021999189900351>.
- [4] Aftosmis MJ, Berger MJ, Melton JE. Robust and efficient cartesian mesh generation for component-based geometry. *AIAA J* 1998;36(6):952–60. <http://dx.doi.org/10.2514/2.464>.
- [5] Aftosmis M. Solution adaptive cartesian grid methods for aerodynamics flows with complex geometries. Technical Report FRBF-0001, von Karman Institute for Fluid Dynamics; 1997, Lecture Series 1997-02.
- [6] Berger M, LeVeque R. Adaptive mesh refinement using wave-propagation algorithms for hyperbolic systems. *SIAM J Numer Anal* 1998;35(6):2298–316. <http://dx.doi.org/10.1137/S0036142997315974>, URL <http://epubs.siam.org/DOI/abs/10.1137/S0036142997315974>, arXiv: <http://epubs.siam.org/DOI/pdf/10.1137/S0036142997315974>.
- [7] Aftosmis M, Berger M, Adomavicius G. A parallel multilevel method for adaptively refined cartesian grids with embedded boundaries. In: 38th Aerospace sciences meeting and exhibit. 2000, p. 808.
- [8] Aftosmis M, Berger M, Murman S. Applications of space-filling-curves to cartesian methods for cfd. In: 42nd AIAA aerospace sciences meeting and exhibit. 2004, p. 1232.
- [9] Henry de Frahan MT, Rood JS, Day MS, Sitaraman H, Yellapantula S, Perry BA, Grout RW, Almgren A, Zhang W, Bell JB, Chen JH. Pelec: An adaptive mesh refinement solver for compressible reacting flows. *Int J High Perform Comput Appl* 2022;10943420221121151. <http://dx.doi.org/10.1177/10943420221121151>.
- [10] Hoppe N, Winter JM, Adami S, Adams NA. ALPACA - A level-set based sharp-interface multiresolution solver for conservation laws. *Comput Phys Comm* 2022;272:108246. <http://dx.doi.org/10.1016/j.cpc.2021.108246>.
- [11] Freret L, Ivan L, De Sterck H, Groth CP. A high-order finite-volume method with anisotropic AMR for ideal MHD flows. In: 55th AIAA aerospace sciences meeting. 2017, p. 0845.
- [12] Freret L, Ivan L, De Sterck H, Groth CP. High-order finite-volume method with block-based AMR for magnetohydrodynamics flows. *J Sci Comput* 2019;79(1):176–208.

- [13] Owen L, Guzik S, Gao X. A high-order adaptive algorithm for multispecies gaseous flows on mapped domains. *Comput & Fluids* 2018;170:249–60.
- [14] Peskin CS. Flow patterns around heart valves: A numerical method. *J Comput Phys* 1972;10(2):252–71. [http://dx.doi.org/10.1016/0021-9991\(72\)90065-4](http://dx.doi.org/10.1016/0021-9991(72)90065-4), URL <https://www.sciencedirect.com/science/article/pii/0021999172900654>.
- [15] Peskin CS. Numerical analysis of blood flow in the heart. *J Comput Phys* 1977;25(3):220–52.
- [16] Peskin CS. The immersed boundary method. *Acta Numer* 2002;11:479–517. <http://dx.doi.org/10.1017/S0962249202000077>.
- [17] Kim W, Choi H. Immersed boundary methods for fluid-structure interaction: A review. *Int J Heat Fluid Flow* 2019.
- [18] Seshadri PK, De A. A novel sharp interface immersed boundary framework for viscous flow simulations at arbitrary mach number involving complex and moving boundaries. *Comput & Fluids* 2020;206:104579. <http://dx.doi.org/10.1016/j.compfluid.2020.104579>, URL <https://www.sciencedirect.com/science/article/pii/S0045793020301511>.
- [19] Di Mascio A, Zaghi S. An immersed boundary approach for high order weighted essentially non-oscillatory schemes. *Comput & Fluids* 2021;222:104931.
- [20] Cook S. CUDA programming: A developer's guide to parallel computing with GPUs. first ed.. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.; 2012.
- [21] NVIDIA, Vingelmann P, Fitzek FH. CUDA, release: 11.8. 2022, URL <https://developer.nvidia.com/cuda-toolkit>.
- [22] AMD. AMD ROCm open ecosystem, release: 5.3. 2022, URL <https://docs.amd.com/>.
- [23] Intel. Intel oneapi, release: 2022.3. 2022, URL <https://docs.oneapi.io/versions/latest/index.html>.
- [24] Chandra R, Dagum L, Kohr D, Menon R, Maydan D, McDonald J. Parallel programming in openMP. Morgan kaufmann; 2001.
- [25] OpenMP Architecture Review Board. OpenMP application program interface version 5.2. 2021, URL <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>.
- [26] OpenACC Architecture Review Board. OpenACC application program interface version 3.1. 2020, URL <https://www.openacc.org/sites/default/files/inline-images/Specification/OpenACC-3.1-final.pdf>.
- [27] Weller H, Tabor G, Jasak H, Fureby C. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Comput Phys* 1998;12(6):620–31.
- [28] Bernardini M, Modesti D, Salvatore F, Pirozzoli S. STREAMS: A high-fidelity accelerated solver for direct numerical simulation of compressible turbulent flows. *Comput Phys Comm* 2021;263:107906. <http://dx.doi.org/10.1016/j.cpc.2021.107906>.
- [29] Witherden FD, Farrington AM, Vincent PE. PyFR: An open source framework for solving advection-diffusion type problems on streaming architectures using the flux reconstruction approach. *Comput Phys Comm* 2014;185(11):3028–40.
- [30] Paul F, Fischer JWL, Kerkemeier SG. nek5000 web page. 2008, <http://nek5000.mcs.anl.gov>.
- [31] Morton GM. A computer oriented geodetic data base and a new technique in file sequencing. In: IBM Germany scientific symposium series. International Business Machines Company; 1966.
- [32] Lorton K, Wise D. Analyzing block locality in morton-order and morton-hybrid matrices. *SIGARCH Comput Archit News* 2007;35(4):6–12. <http://dx.doi.org/10.1145/1327312.1327315>, URL <http://DOI.acm.org/10.1145/1327312.1327315>.
- [33] Kleine H, Ritzfeld E, Grönig H. Shock wave diffraction—new aspects of an old problem. In: Shock waves @marseille IV. Springer; 1995, p. 117–22.
- [34] Kleine H, Grönig H, Takayama K. Simultaneous shadow, schlieren and interferometric visualization of compressible flows. *Opt Lasers Eng* 2006;44(3):170–89.
- [35] Yeom G-S, Chang K-S. Shock wave diffraction about a wedge in a gas-microdroplet mixture. *Int J Heat Mass Transfer* 2010;53(23):5073–88. <http://dx.doi.org/10.1016/j.jheatmasstransfer.2010.07.056>, URL <https://www.sciencedirect.com/science/article/pii/S0017931010004370>.
- [36] Gnani F, Lo K, Zare-Behtash H, Kontis K. Experimental investigation on shock wave diffraction over sharp and curved splitters. *Acta Astronaut* 2014;99:143–52. <http://dx.doi.org/10.1016/j.actaastro.2014.02.018>, URL <https://www.sciencedirect.com/science/article/pii/S0094576514000757>.
- [37] Zhang L, Wang T, Hao L, Huang B, Chen W, Shi H. Experimental investigation of shock wave diffraction over a single-or double-sphere model. *Shock Waves* 2017;27(1):97–107.
- [38] Brahmi N, Hadjadj A, Soni V, Chaudhuri A. Analysis of shock-wave diffraction over double concave cylindrical wedges. Part I: Shock dynamics. *Acta Astronaut* 2020;172:134–9. <http://dx.doi.org/10.1016/j.actaastro.2020.01.025>, URL <https://www.sciencedirect.com/science/article/pii/S0094576520300369>.
- [39] Brahmi N, Hadjadj A, Soni V, Chaudhuri A. Analysis of shock-wave diffraction over double cylindrical wedges. Part II: Vorticity generation. *Acta Astronaut* 2020;172:140–50. <http://dx.doi.org/10.1016/j.actaastro.2020.02.017>, URL <https://www.sciencedirect.com/science/article/pii/S0094576520300795>.
- [40] Tsai I-N, Huang J-C, Tsai S-S, Yang J. Unsteady relativistic shock-wave diffraction by cylinders and spheres. *Phys Rev E* 2012;85(2):026317.
- [41] Mehta Y, Jackson T, Zhang J, Balachandrar S. Numerical investigation of shock interaction with one-dimensional transverse array of particles in air. *J Appl Phys* 2016;119(10):104901.
- [42] Martyushov S, Igra O, Elperin T. Numerical investigation of shock wave diffraction over a sphere placed in a shock tube. *Int J Aerosp Eng* 2016;2016.
- [43] Bryson A, Gross R. Diffraction of strong shocks by cones, cylinders, and spheres. *J Fluid Mech* 1961;10(1):1–16.
- [44] Navier C. Mémoire sur les lois du mouvement des fluides. *Mém l'Acad R Sci l'Inst France* 1822;6:389–440.
- [45] Stokes G. Report on recent researches in hydrodynamics. *Br. Assoc Adv Sci* 1846;1:1–20.
- [46] Craik A. The origins of water wave theory. *Annu Rev Fluid Mech* 2004;36(1):1–28. <http://dx.doi.org/10.1146/annurev.fluid.36.050802.122118>, URL <http://www.annualreviews.org/DOI/abs/10.1146/annurev.fluid.36.050802.122118>.
- [47] Craik A. George gabriel Stokes on water wave theory. *Annu Rev Fluid Mech* 2005;37(1):23–42. <http://dx.doi.org/10.1146/annurev.fluid.37.061903.175836>, URL <http://www.annualreviews.org/DOI/abs/10.1146/annurev.fluid.37.061903.175836>.
- [48] Lebedev V. The equations and convergence of a differential-difference method (the method of lines). *Vestnik Moskovskogo Gosudarstvennogo Univ* 1955;10:47–57.
- [49] Jiang G, Shu C. Efficient implementation of weighted ENO schemes. *J Comput Phys* 1996;126(1):202–28. <http://dx.doi.org/10.1006/jcph.1996.0130>, URL <http://www.sciencedirect.com/science/article/pii/S002199919601308>.
- [50] Shu C, Osher S. Efficient implementation of essentially non-oscillatory shock-capturing schemes. *J Comput Phys* 1988;77(2):439–71.
- [51] Liu X, Osher S, Chan T. Weighted essentially non-oscillatory schemes. *J Comput Phys* 1994;115(1):200–12. <http://dx.doi.org/10.1006/jcph.1994.1187>, URL <http://www.sciencedirect.com/science/article/pii/S0021999184711879>.
- [52] Gerolymos G, Sénéchal D, Vallet I. Very-high-order WENO schemes. *J Comput Phys* 2009;228(23):8481–524.
- [53] Lax P. Weak solutions of nonlinear hyperbolic equations and their numerical computation. *Comm Pure Appl Math* 1954;7(1):159–93.
- [54] Rusanov VV. Calculation of intersection of non-steady shock waves with obstacles. *J Comput Math Phys* 1961;1:267–79.
- [55] Harten A, Lax P, van Leer B. On upstream differencing and godunov-type schemes for hyperbolic conservation laws. *SIAM Rev* 1983;25(1):35–61. <http://dx.doi.org/10.1137/1025002>, URL <http://epubs.siam.org/DOI/abs/10.1137/1025002>.
- [56] Runge C. Ueber die numerische Auflösung von Differentialgleichungen. *Math Ann* 1895;46(2):167–78. <http://dx.doi.org/10.1007/BF01446807>.
- [57] Kutta W. Beitrag zur näherungsweise integration totaler differentialgleichungen. *Z Angew Math Phys* 1901;46:435–53.
- [58] Gottlieb S, Shu C, Tadmor E. Strong stability-preserving high-order time discretization methods. *SIAM Rev* 2001;43:89–112.
- [59] Butcher J. Numerical methods for ordinary differential equations. Second ed.. Wiley; 2008, p. 482.
- [60] Gottlieb S, Ketcheson DI, Shu C-W. High order strong stability preserving time discretizations. *J Sci Comput* 2009;38(3):251–89.
- [61] Bresten C, Gottlieb S, Grant Z, Higgs D, Ketcheson D, Nemeth A. Explicit strong stability preserving multistep Runge–Kutta methods. *Math Comp* 2014;86(304):747–69.
- [62] Gottlieb S, Ketcheson DI. Time discretization techniques. In: *Handbook of numerical analysis*, Vol. 17. Elsevier; 2016, p. 549–83.
- [63] Hadjimihael Y, Ketcheson DI, Lóczy L, Nemeth A. Strong stability preserving explicit linear multistep methods with variable step size. *SIAM J Numer Anal* 2016;54(5):2799–832.
- [64] Ranocha H, Sayyari M, Dalcin L, Parsani M, Ketcheson DI. Relaxation Runge–Kutta methods: Fully discrete explicit entropy-stable schemes for the compressible Euler and Navier–Stokes equations. *SIAM J Sci Comput* 2020;42(2):A612–38.
- [65] Nüßlein S, Ranocha H, Ketcheson DI. Positivity-preserving adaptive Runge–Kutta methods. *Commun Appl Math Comput Sci* 2021;16(2):155–79.
- [66] Kuzmin D, Quezada de Luna M, Ketcheson DI, Grüll J. Bound-preserving flux limiting for high-order explicit Runge–Kutta time discretizations of hyperbolic conservation laws. *J Sci Comput* 2022;91(1):1–34.
- [67] Ranocha H, Dalcin L, Parsani M, Ketcheson DI. Optimized runge-kutta methods with automatic step size control for compressible computational fluid dynamics. *Commun Appl Math Comput* 2022;4(4):1191–228.
- [68] Courant R, Friedrichs K, Lewy H. Über die partiellen differenzengleichungen der mathematischen physik. *Math Ann* 1928;100(1):32–74. <http://dx.doi.org/10.1007/BF01448839>.
- [69] Sethian JA, Smereka P. Level set methods for fluid interfaces. *Annu Rev Fluid Mech* 2003;35(1):341–72. <http://dx.doi.org/10.1146/annurev.fluid.35.101101.161105>, URL <http://www.annualreviews.org/DOI/abs/10.1146/annurev.fluid.35.101101.161105>.
- [70] Poinso T, Lele S. Boundary conditions for direct simulations of compressible viscous flows. *J Comput Phys* 1992;101(1):104–29.